

Final! robby - hope to get more work done.

Q. What is weight and bias?

⇒ Weight: is determines the importance of an input in a neural network.

• Each input has a weight.
• Higher weight = greater influence on output.

• Weights are learned during training.

Example If $x=2$ and $w=0.5$, then

$$\text{contribution} = 2 \times 0.5 = 1$$

Bias: is an additional value added to the weighted sum to adjust the output.

• Helps the model fit data better.
• Similar to the intercept in a linear equation.

• Allows output shifting independent of inputs.

$$\text{formula } y = \sum w_i x_i + b$$

Q. What is forward propagation?

⇒ Forward Propagation is the process of passing input data through a neural network to calculate the output.

- Data flows from input layer $\rightarrow$ hidden layer (s) $\rightarrow$ output layer.

- Uses weights and biases to compute outputs.
- Activation functions are applied at each neuron.
- It is the first step in training and prediction.

Example $\rightarrow$ convolutional neural network = feature extractor

Input image $\rightarrow$ Neural network $\rightarrow$ Predicted label(s)

* Stemming:

is a text preprocessing technique that reduces words to obtain root form (stem). (removing suffix and prefixes).

- Used in NLP
- Helps reduce vocabulary size.
- Different forms of a word are treated as the same word.

Examples:

playing $\rightarrow$ play
played $\rightarrow$ play
plays, played $\rightarrow$ play

Q. Problems in stemming -

- $\rightarrow$ May generate non-dictionary words.
- Can remove too many characters (over-stemming)
- Can fail to reduce related words to the same stem (under-stemming)

- May lose the original meaning of words.

Example 1

University $\rightarrow$ Univers (incorrect stem)

* Better and good stemming are related but not reduced to the same stem.

Q. Advantages and disadvantages of stemming.

$\Rightarrow$

Advantages	Disadvantages
1. Reduces vocabulary size.	1. May produce meaningless root words.
2. Improves text processing speed.	2. Can cause over-stemming.
3. Saves storage and memory.	3. Can cause under-stemming.
4. Helps group similar words together.	4. May lose the original meaning of words.
5. Useful for search engines and NLP tasks.	5. Reduces accuracy in some applications.

Q. What is NLP?

⇒ Natural language processing is a branch of AI that enables computers to understand, process, and generate human language. It helps machines interact with text and speech data.

Q. NLP Applications:

⇒ NLP used in

- chatbots,
- machine translation,
- sentiment analysis.
- speech recognition.
- spam detection.

It helps automate language-related tasks.

Q. What is Tokenization?

⇒ It is the process of breaking text into smaller units called tokens, such as words or sentences. It is usually the first step in NLP preprocessing.

Q. What is Stop Word Removal?

⇒ It is the process of removing common words such as "the", "is", "and" from text. This helps reduce unnecessary data and improve processing efficiency.

Q. Can we always use stop word removal before stemming?

⇒ No, stop word removal is not always necessary before stemming. In some tasks, stop words carry important information and removing them may reduce accuracy.

Q. What is Lemmatization?

⇒ It is the process of converting words to their dictionary base form (lemma) using vocabulary and grammar rules. For example, "running" becomes "run" and "better" becomes "good".

Q. Differences between Lemmatization and Stemming.

Stemming	Lemmatization
1. Removes prefixes/suffixes to get the root word.	1. Converts a word to its dictionary-based form (lemma).
2. Uses simple rule-based cutting.	2. Uses vocabulary and grammatical analysis.
3. Faster and computationally cheaper.	3. Slower and more computationally expensive.
4. May produce non-meaningful root words.	4. Produces meaningful words.
5. Less accurate.	5. More accurate.
6. Ex: Studier → Studi	6. Ex: Studier → Study.

a. Advantages and disadvantages of both.

⇒ Stemming

Advantages	Disadvantages
1. Faster and computationally efficient.	1. May produce meaningless words.
2. Reduces vocabulary size for faster processing.	2. Less accurate than lemmatization

Lemmatization:

Advantages	Disadvantages
1. Produces meaningful dictionary words.	1. Slower than stemming.
2. More accurate and preserves word meaning.	2. Requires more computational resources.

a. Definition of corpus, vocabulary, document word.

⇒ Corpus: is a collection of documents or text data

used for NLP tasks. It serves as the dataset for training and analyzing language models.

Vocabulary: is the set of all unique words present in a corpus. It represents the words that

model can recognize or process.

Document: is a single unit of text within a corpus, such as an article, email, or review. A corpus consists of one or more documents.

Word: is the smallest meaningful unit of text used in NLP. Words are typically obtained after tokenization and form the building blocks of documents.

⇒ One-hot encoding:

is a technique used to convert categorical data on words into binary vectors. In the vector, only one position is 1 and all other positions are 0.

Example: Vocabulary = {cat, Dog, Bird}

cat ⇒ [1, 0, 0]

Dog ⇒ [0, 1, 0]

Bird ⇒ [0, 0, 1]

*

D1	Student read DL
D2	DL read DL
D3	Student write comment
D4	DL write comment.

Compare Student read DL

DL read DL
Student write comment

DL write comment
Vocabulary: Student, read DL write comment

$D=5$

DL Vectors $[1, 0, 0, 0, 0]$, $[0, 1, 0, 0, 0]$, $[0, 0, 1, 0, 0]$

Dimension: $[3, 5]$ vocabulary $= 5$

Advantages and disadvantages of One-hot encoding

Advantages	Disadvantages
1. Simple and easy to implement.	1. Creates high-dimensional vectors for large vocabularies
2. No numerical relationship is assumed between categories	2. Produces sparse vectors (mostly zeros)
3. Works well for basic machine learning models.	3. Cannot capture semantic relationships between words

Bag of Words (BoW)

$\Rightarrow$ is a text representation technique that converts text into numerical vectors based on the frequency of words. It ignores grammar and word order, considering only the occurrence of words.

DL	Student	read	DL	1
DL	DL	read	DL	But
DL	student	write	comment	0
DL	DL	DL	comment	DL
DL	DL	DL	comment	DL

Vocabulary: Student read DL write comment But

Doc	Student	read	DL	write	comment	But
DL	1	1	1	0	0	0
DL	0	0	4	1	1	0

Advantages and disadvantages of BoW

Advantages	Disadvantages
1. Simple and easy to implement.	1. Ignores word order and context.
2. Converts text into numerical form for machine learning.	2. Creates high-dimensional feature vectors.
3. Easy to train and process.	3. Produces sparse matrices

with many zeros

4. Effective for basic

4. Cannot capture semantic meaning

text classification tasks

between words

TF-IDF:

is a text representation technique that measures how important a word is in a document relative to a collection of documents (corpus). It gives higher weight to important words and lower weight to very common words.

Term frequency

Inverse Document Frequency

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t) = \frac{\text{total number of documents in the corpus}}{\text{Number of documents with term } t \text{ in them}}$$

TF-IDF table

D1: Student watch mobile	1
D2: Mobile watch student Screen	1
D3: Student write comment	0
D4: Screen write comment	0

TF table:

Term	D1	D2	D3	D4
Student	1/3	0	1/3	0
Watch	1/3	1/3	0	0
Mobile	1/3	1/3	0	0
Screen	0	1/3	0	1/3
Write	0	0	1/3	1/3
Comment	0	0	1/3	1/3

Vocabulary:

Student, Watch, Mobile, Screen, write, comment

TF-IDF table: Total Doc, $N=4$, Each term appears in 2 documents

Term	D1	D2	D3	D4
Student	$\frac{1}{3} \log \frac{4}{2}$	0	$\frac{1}{3} \log \frac{4}{1}$	0
Watch	$\frac{1}{3} \log \frac{4}{2}$	$\frac{1}{3} \log \frac{4}{2}$	0	0
Mobile	$\frac{1}{3} \log \frac{4}{2}$	$\frac{1}{3} \log \frac{4}{2}$	0	0
Screen	0	$\frac{1}{3} \log \frac{4}{2}$	0	$\frac{1}{3} \log \frac{4}{2}$
Write	0	0	$\frac{1}{3} \log \frac{4}{2}$	$\frac{1}{3} \log \frac{4}{2}$
Comment	0	0	$\frac{1}{3} \log \frac{4}{2}$	$\frac{1}{3} \log \frac{4}{2}$

Word Embedding:

is a technique that converts words into dense numerical vectors while preserving their semantic meaning. Similar words are represented by vectors that are close to each other in the vector space.

Example: Words like "King" and "Queen" will have similar vector representations because they are semantically related.

Types of Word Embedding

Frequency Based

→ One-hot encoding

→ Bag of Words (BoW)

→ TF-IDF

Prediction Based

→ Word2Vec

→ GloVe (Continuous BoW)

→ Skip-gram

→ GloVe

→ FastText

Q. Why we need word embedding?

⇒ Word embedding is needed to convert words into meaningful numerical vectors that computers can process. It captures the semantic relationships

between words, unlike 'one-hot encoding' on BoW.

Key points:

- Reduces high-dimensional sparse vectors.
- Captures word meaning and context.
- Improves performance of NLP and DL models.

Q. Advantages and disadvantages of TF-IDF.

Advantages	Disadvantages
1. Highlights important words in a document.	1. Ignores word order and context.
2. Reduces the impact of common words.	2. Cannot capture semantic meaning.
3. Simple and easy to implement.	3. Creates high-dimensional sparse vectors.
4. Works well for text classification and information retrieval.	4. Performance decreases with very large vocabularies.

Embedding means finding

	King	Queen	Man	Woman	Monkey
Gender	1	0	1	0	0
Wealth	1	1	0.3	0.3	0.08
Power	1	0.7	0.2	0.2	0.002
Weight	0.8	0.4	0.6	0.59	0.3
Speak	1	1	1	1	0

Word 2Vec: $\text{Man} - \text{Woman} = \text{King} - \text{Queen}$

$\text{Man} = [0.5, 0.6]$ $\text{Woman} = [0.5, 0.6]$

$\text{King} = [0.5, 0.62]$ $\text{Queen} = [0.5, 0.62]$

$\text{King} = [1, 1, 1, 0.8, 1]$

$\text{Queen} = [0, 1, 0.7, 0.4, 1]$

$\text{Man} = [1, 0.3, 0.2, 0.6, 1]$

$\text{Woman} = [0, 0.3, 0.2, 0.5, 1]$

$\text{Monkey} = [1, 0, 0, 0.3, 0]$

King - Man + Woman: $[0, 1, 1, 0.7, 1]$

Almost Queen Embedding

Cosine Similarity:

is a measure used to determine how similar two vectors on documents are by calculating the cosine of the angle between them. Its value ranges from -1 to 1, where 1 means identical and 0 means no similarity.

Formula:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{|A| |B|}$$

Applications:

• Document Similarity

• Search engines

• Recommendation systems

Steps to work:

1. create TF table + TF-IDF table
2. Query column add
3. Use cosine similarity theorem
4. Rank Documents

Example:

- ① Student watch mobile
 - ② Mobile watch screen
 - ③ Student write comment
 - ④ Screen write comment
- for this sentence:
"Student watch watch mobile screen"
- find cosine similarity.

TF-table: total document = 4

Vocabulary = {Student, watch, mobile, screen, write, comment}

Term	Doc1	Doc2	Doc3	Doc4	Q	TFIDF(t) = $\log(\frac{N}{df_t})$
Student	1/3	0	1/3	0	1	$\log \frac{4}{2} = 0.3$
Watch	1/3	1/3	0	0	2	$\log \frac{4}{2} = 0.3$
Mobile	1/3	1/3	0	0	1	$\log \frac{4}{2} = 0.3$
Screen	0	0	1/3	1/3	0	$\log \frac{4}{2} = 0.3$
Write	0	1/3	0	1/3	1	$\log \frac{4}{2} = 0.3$
Comment	0	0	1/3	1/3	0	$\log \frac{4}{2} = 0.3$

TF-IDF table

Term	Doc1	Doc2	Doc3	Doc4	Q
Student	$1/3 \times \log \frac{4}{2} = 0.1$	0	0.1	0	$1 \times 0.3 = 0.3$
Watch	0.1	0.1	0	0	0.6
Mobile	0.1	0.1	0	0	0.3
Write	0	0	0.1	0.1	0
Screen	0	0.1	0	0.1	0.3
Comment	0	0.1	0.1	0.1	0

cosine similarity = $\frac{D \cdot Q}{|D| \cdot |Q|}$

$$|D| = \sqrt{(0.3)^2 + (0.1)^2 + (0.3)^2 + (0.3)^2} = 0.79$$

$$|D_1| = \sqrt{(0.1)^2 + (0.1)^2 + (0.1)^2} = 0.57$$

$$|D_2| = \sqrt{(0.1)^2 + (0.1)^2 + (0.1)^2} = 0.57$$

$$|D_3| = \sqrt{(0.1)^2 + (0.1)^2 + (0.1)^2} = 0.57$$

$$|D_4| = \sqrt{(0.1)^2 + (0.1)^2 + (0.1)^2} = 0.57$$

$$\therefore CS(D, D_1) = \frac{0.1 \times 0.3 + 0.1 \times 0.3 + 0.1 \times 0.3}{0.79 \times 0.57} = 0.89$$

$$CS(D, D_2) = \frac{0.1 \times 0.3 + 0.1 \times 0.3 + 0.1 \times 0.3}{0.79 \times 0.57} = 0.89$$

$$CS(D, D_3) = 0.22$$

$$CS(D, D_4) = 0.22$$

Relevant : Answer

Continuous Bag of Words

is a Word2Vec model that predicts a target using its surrounding context words. It learns word embeddings by analyzing neighboring words.

Example 1.0

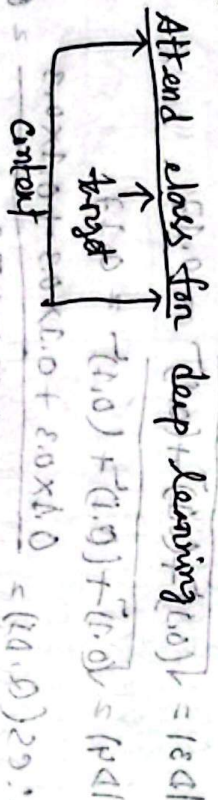
Sentence: "I love deep learning."

Context: "I, love, learning" → Predict: deep

Key points:

- Predicts the center word from context words.
- Faster to train than skip-gram.
- Works well with frequent words.

False/dummy problem → (solution) = 1.5



context word

Attend, for

class, deep

for, learning

target word

class

for

deep

target word

class

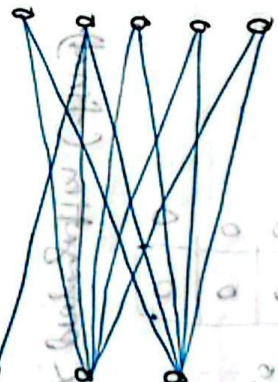
for

deep

Attend class for deep learning.

Just: 10000

every



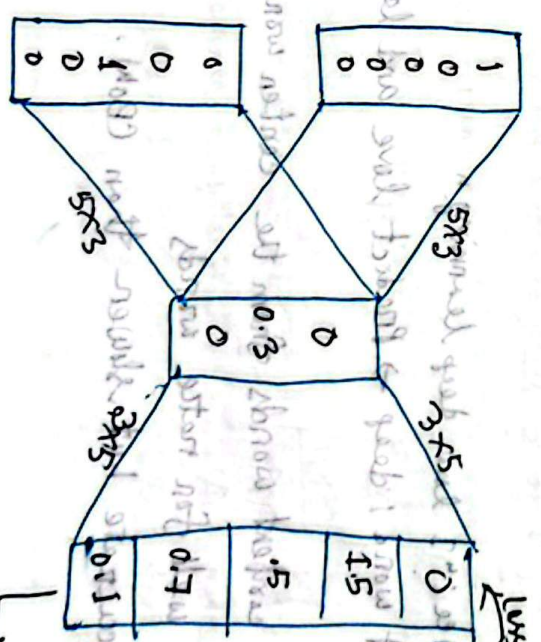
Note:

Three point here because if three feature available

Neural Network

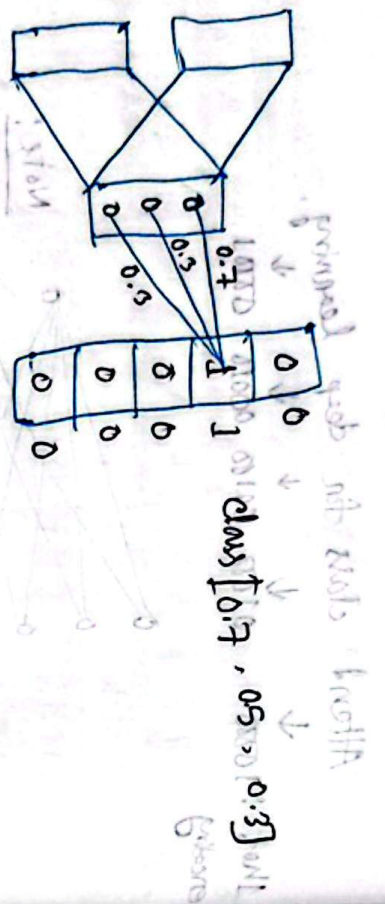
context word

target word



Let's calculate

→ Target word
→ Predicted word



After forward-back propagation (find)

Skip-Gram

is a word2vec model that predicts the surrounding context words from a given target word. It learns word embeddings by using the target word to predict its neighbors.

Example:

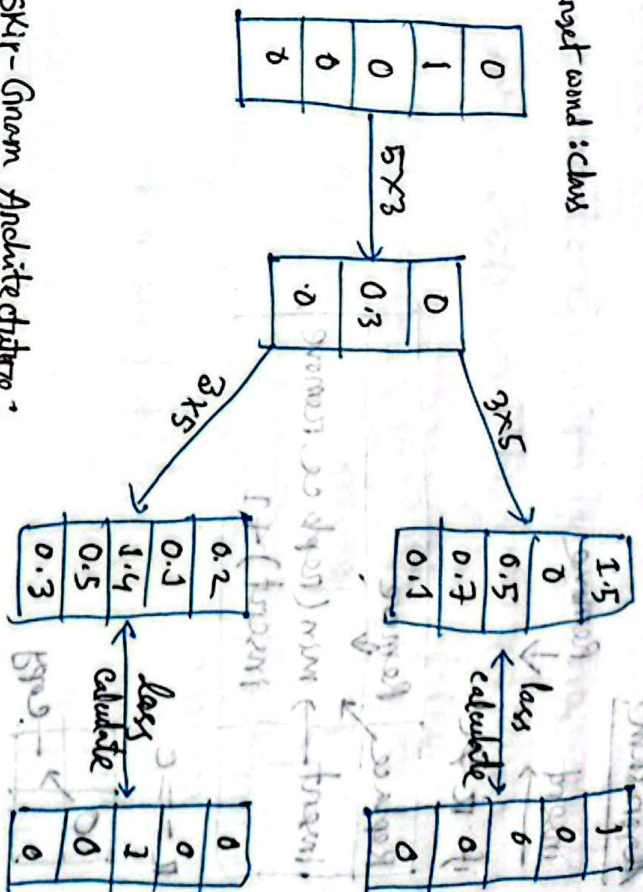
Sentence: "I love deep learning"

Target word: "love" → Predict "love" and learning

Key points:

- Predicts context words from the center word.
- Works well for rare words
- More accurate but slower than CBOW.

Target word: class



Skip-Gram Architecture:

Minimum Edit Distance (MED)

is the minimum number of operation required to transform one string into another. The allowed operations are, insertion, deletion, substitution.

Example:

"eat" → "cut"

Replace a with u

MED = 1

Applications:

- Spell checking
- Auto correction
- NLP text modeling

Algorithm:

insert $\rightarrow$ Remove
if $r \neq c$

Replace	Remove
insert $\rightarrow$ min(replace, remove)	insert $\rightarrow +1$

$r = c$

0	1
1	2

copy

0	1	0
0	0	0

Example: adceq $\rightarrow$ abctfg

	a	b	c	f	g
a	0 $\rightarrow$ 1	1 $\rightarrow$ 2	2 $\rightarrow$ 3	3 $\rightarrow$ 4	4 $\rightarrow$ 5
d	1 $\rightarrow$ 2	2 $\rightarrow$ 3	3 $\rightarrow$ 4	4 $\rightarrow$ 5	5 $\rightarrow$ 6
e	2 $\rightarrow$ 3	3 $\rightarrow$ 4	4 $\rightarrow$ 5	5 $\rightarrow$ 6	6 $\rightarrow$ 7
q	3 $\rightarrow$ 4	4 $\rightarrow$ 5	5 $\rightarrow$ 6	6 $\rightarrow$ 7	7 $\rightarrow$ 8

Answer is 2, 2

Translate:

is the automatic conversion of text from one language to another while preserving its meaning.

Ex: ① Think about it $\rightarrow$ after find the or

② Come in $\rightarrow$ find the answer



Think about it

Seq2Seq model:

is a deep learning model that converts one sequence into another sequence. It consists of an Encoder and a Decoder, where the encoder processes the input sequence and the decoder generates the output sequence.

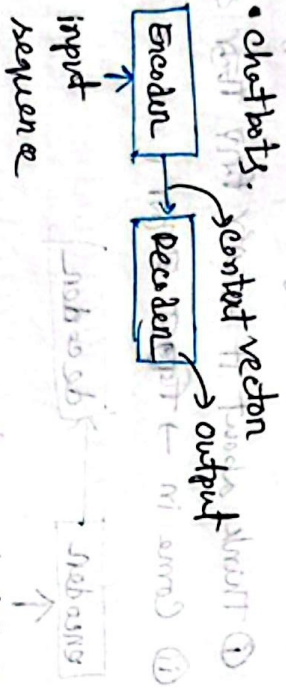
Ex: Eng: "I am a student."

Seq 2 Seq model

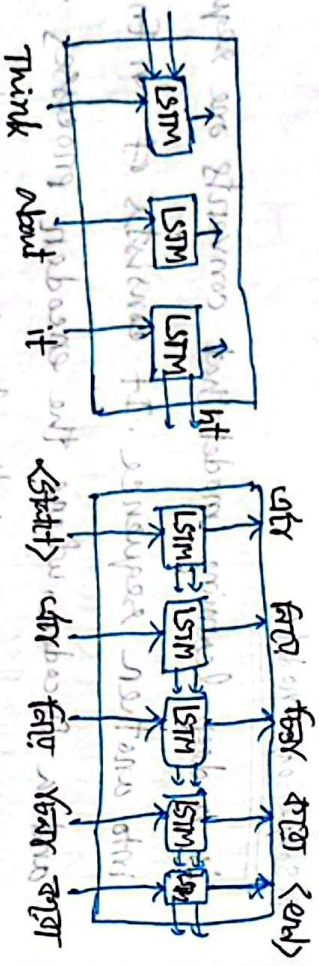
Don't forget to add the

Applications:

- Machine translation
- Text Summarization
- Chatbots



Encoder consists a lot of LSTM model, decoder also.



Encoder side

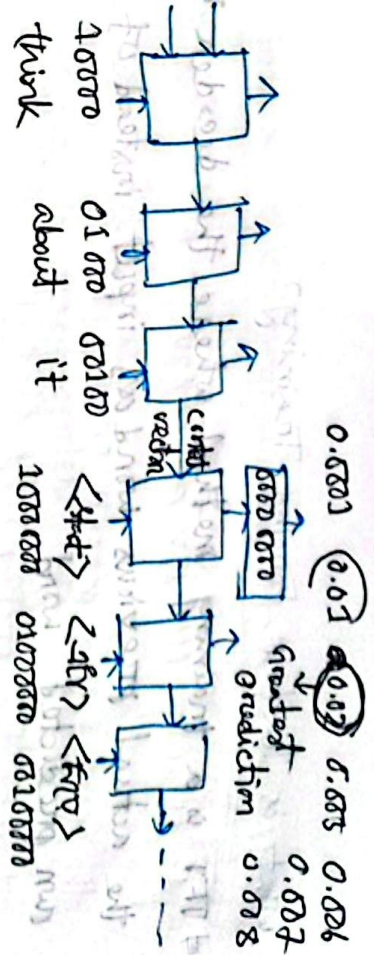
Decoder side

Training phase 1

Think about it → $\langle \text{start} \rangle$ after after after after

Think 1-hot encoding → 10000

for 1 4 5 6 7 8 9 10

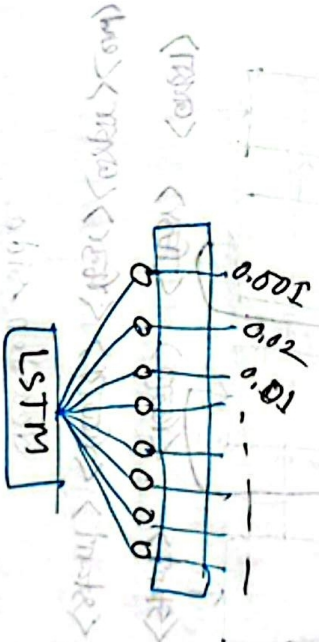


Encoder

Decoder

$$y = [0.100000, [0.000000], [0.000000], [0.000000]]$$

$$y = [0.001, 0.001, \dots, [0.1, \dots], [0.1, \dots]]$$



Forward

Loss calculate → Back Propagation

update weights with optimizer

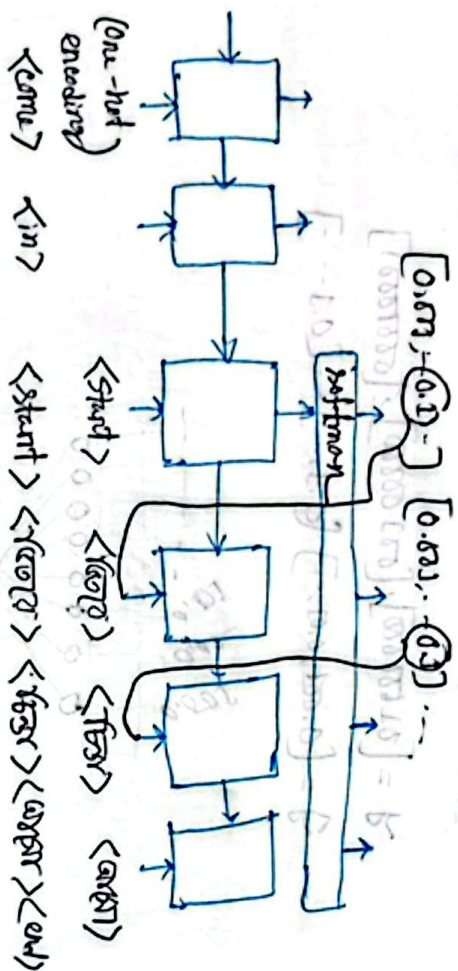
A.F, w, b update

summarizing all weights of layers and

Q. What is Teacher Forcing Training?

→ TFT is a training method where the decoder receives the actual previous word as input instead of its own predicted word.

* Testing Phase:



encoder side

decoder side

Q. How to improve seq2seq model?

→ The basic seq2seq model struggles with long sentences because the entire input must be compressed into a single context vector. Several techniques are used to improve its performance.

1. Attention Mechanism:

- Allows the decoder to focus on important parts of the sentence.

- Solves the fixed context vector problem.

2. Bidirectional LSTM:

- Reads the input sequence in both forward and backward directions.

- Captures more context.

3. Beam Search:

- Considers multiple possible outputs instead of choosing only the most probable word at each step.

- Produces better translations.

4. Transformer Model:

- Replaces LSTMs with self-attention.

- Faster and more accurate than traditional seq2seq.

seq2seq.

ERN-Gram Model!

An N-Gram Model is a language model that predicts the next word based on the previous N-1 words.

It uses the probability of word sequences from

a corpus.

Example:

Sentence: "I love Deep Learning."

Applications:

- ① Next-word prediction
- ② Auto complete
- ③ Machine translation.
- ④ Speech recognition.

Types:

- ① Unigrams $\rightarrow$ no history used
- ② Bigrams $\rightarrow$ one history used
- ③ Trigrams $\rightarrow$ two history used.

Example:

Sentence: "I love Deep Learning."

- Unigram: {I, love, Deep, Learning}
- Bigram: {I, love, love Deep, Deep Learning}
- Trigram: {I love Deep, love Deep Learning}

conditional probability is the probability of an

event occurring given that another event has already occurred. In NLP, it is used to predict a word based on previous words.

formula:
$$P(B|A) = \frac{P(A,B)}{P(A)}$$

Example:
$$P(A,B) = P(A) \cdot P(B|A)$$

Example:
$$P(A,B,C,D) = P(A) \cdot P(B|A) \cdot P(C|A,B) \cdot P(D|A,B,C)$$

Example:
$$P(x_1, x_2, x_3) = P(x_1) \cdot P(x_2|x_1) \cdot P(x_3|x_1, x_2)$$

Example:
$$P(\text{Think about it}) = P(\text{Think}) \cdot P(\text{about Think}) \cdot P(\text{this Think about})$$

Example:

"about (five, min, from, ..."

Unigram: no word history.

$$P(w_i) = \prod_{i=1}^N \frac{\text{count}(w_i)}{N}$$

$$(1, 2, 3, 4, 5)$$

$$P(\text{college} | \text{about, five, min, from}) = P(\text{college}) = \frac{\text{count}(\text{college})}{N}$$

$$P(\text{home} | \text{about, five, min, from}) = P(\text{home}) = \frac{\text{count}(\text{home})}{N}$$

Unigram distribution:

$$\text{count}(\text{about, five, min, from}) = P(\text{about}) \times P(\text{five}) \times P(\text{min}) \times P(\text{from})$$

$$\text{count}(\text{about, five, min, from, college}) = P(\text{about}) \times P(\text{five}) \times P(\text{min}) \times P(\text{from}) \times P(\text{college})$$

$$\text{count}(\text{about, five, min, from, home}) = P(\text{about}) \times P(\text{five}) \times P(\text{min}) \times P(\text{from}) \times P(\text{home})$$

Bigram: One word -> the next.

$$P(w_i, w_{i+1}) = \prod_{i=1}^n P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

$$P(\text{college} | \text{about, five min, from}) = \frac{\text{count}(\text{about, five min, from, college})}{\text{count}(\text{about, five min, from})}$$

$$P(\text{home} | \text{about, five min, from}) = \frac{\text{count}(\text{about, five min, from, home})}{\text{count}(\text{about, five min, from})}$$

$$\text{count}(\text{about five min from}) = P(\text{about} | < s >) \cdot P(\text{five} | \text{about}).$$

$$P(\text{min} | \text{five}) \cdot P(\text{from} | \text{min})$$

$$\text{count}(\text{about five min from college}) = P(\text{five} | \text{about}) \cdot P(\text{min} | \text{five}) \cdot P(\text{from} | \text{min}) \cdot P(\text{college} | \text{from})$$

$$\text{count}(\text{about five min from home}) = P(\text{five} | \text{about}) \cdot P(\text{min} | \text{five}) \cdot P(\text{from} | \text{min}) \cdot P(\text{home} | \text{from})$$

Trigram: Two word -> the next.

$$P(w_1, w_2, w_3) = \prod_{i=1}^n P(w_i | w_{i-2}, w_{i-1}) = \frac{\text{count}(w_{i-2}, w_{i-1}, w_i)}{\text{count}(w_{i-2}, w_{i-1})}$$

$$P(\text{college} | \text{about, five min, from}) = \frac{\text{count}(\text{min, from, college})}{\text{count}(\text{min, from})}$$

$$P(\text{home} | \text{about, five min, from}) = \frac{\text{count}(\text{min, from, home})}{\text{count}(\text{min, from})}$$

$$P(\text{about, five min, from}) = P(\text{about} | < s >) \times P(\text{five} | < s > \text{about}) \times P(\text{min} | \text{about, five}) \times P(\text{from} | \text{five, min})$$

$$P(\text{about, five min, from, college}) = P(\text{about, five min, from}) \times P(\text{college} | \text{min, from})$$

$$P(\text{about, five min, from, home}) = P(\text{about, five min, from}) \times P(\text{home} | \text{min, from})$$

Example:

Given corpus:

< s > I am Henry < / s >

< s > I like college < / s >

< s > Do Henry like college < / s >

< s > Henry I am < / s >

< s > Do I like Henry < / s >

< s > Do I like college < / s >

< s > Do like Henry < / s >

Q: < s > Do ?

w_{i-1} = Do

using bigram:

< s > Do ?	P?	(ob do) ?
< s > Do I like Henry < / s >	P?	(ob do) ?
< s > Do I like college < / s >	P?	(ob do) ?
< s > Do Henry like college < / s >	P?	(ob do) ?
< s > Henry I am < / s >	P?	(ob do) ?
< s > Do I like Henry < / s >	P?	(ob do) ?
< s > Do I like college < / s >	P?	(ob do) ?
< s > Do like Henry < / s >	P?	(ob do) ?

Subj: $\langle x \rangle, \langle x \rangle \neq 1$ tokens $g = (\text{most}, \text{win}, \text{swift}, \text{books})$ tokens

Word	Frequency
$\langle x \rangle$	7
$\langle x \rangle$	7
$\langle x \rangle$	7
I	5
am	2
Henry	5
like	5
college	3
do	4

Next word	Probability Next word
	$z = \frac{\text{count}(w_i, w_j)}{\text{count}(w_i - 1)}$
$p(\langle x \rangle \langle x \rangle)$	$9/4$
$p(I \langle x \rangle)$	$2/4$
$p(\text{am} \langle x \rangle)$	$0/4$
$p(\text{Henry} \langle x \rangle)$	$1/4$
$p(\text{like} \langle x \rangle)$	$1/4$
$p(\text{college} \langle x \rangle)$	$0/4$
$p(\text{do} \langle x \rangle)$	$0/4$

I is the most preferable next word.

unigram!

Next word	$P.N.W = \frac{\text{count}(w_j)}{N}$
$p(\langle x \rangle)$	$7/38$
$p(I)$	$5/38$
$p(\text{am})$	$2/38$
$p(\text{Henry})$	$5/38$
$p(\text{like})$	$5/38$
$p(\text{college})$	$3/38$
$p(\text{do})$	$4/38$

$N = \text{total number of tokens}$
 $= 7 + 5 + 2 + 5 + 5 + 3 + 4 = 38$

I will be most preferable, ~~say~~ is structural tags that using this will not count. Ties between "I", "Henry", "like".

Next word	$P.N.W = \frac{\text{count}(w_i, w_{i-1}, w_j)}{\text{count}(w_{i-2}, w_{i-1})}$
$p(\langle x \rangle \langle x \rangle)$	$0/4$
$p(I \langle x \rangle)$	$2/4$
$p(\text{am} \langle x \rangle)$	$0/4$
$p(\text{Henry} \langle x \rangle)$	$1/4$
$p(\text{like} \langle x \rangle)$	$1/4$
$p(\text{college} \langle x \rangle)$	$0/4$
$p(\text{do} \langle x \rangle)$	$0/4$

I most preferable.

Transformer Model:

A transformer is a deep learning architecture that uses self-attention to understand relationships between different parts of input data. Unlike RNNs, it processes the entire sequence simultaneously, making it faster and better at capturing long-range dependencies.

Applications:

- Machine Translation (Google Translate) (e.g.)
- Text Generation (ChatGPT, GPT models)
- Question answering systems.
- Text Summarization.
- Sentiment analysis (e.g., analyzing reviews)
- Speech recognition.
- Image classification (Vision Transformer - ViT)
- Recommendation systems.

Advantages:

1. Parallel processing
2. Captures long range dependencies
3. High performance

4. Scalability. *high*

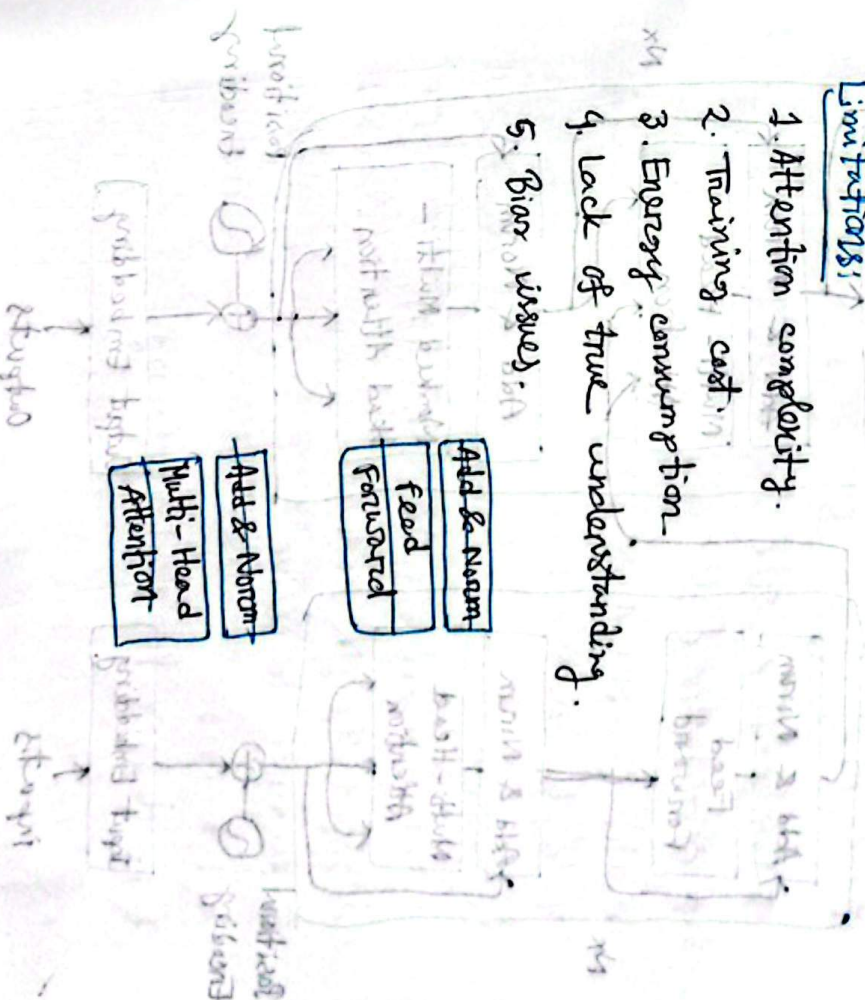
5. Flexible Architecture.

Disadvantages:

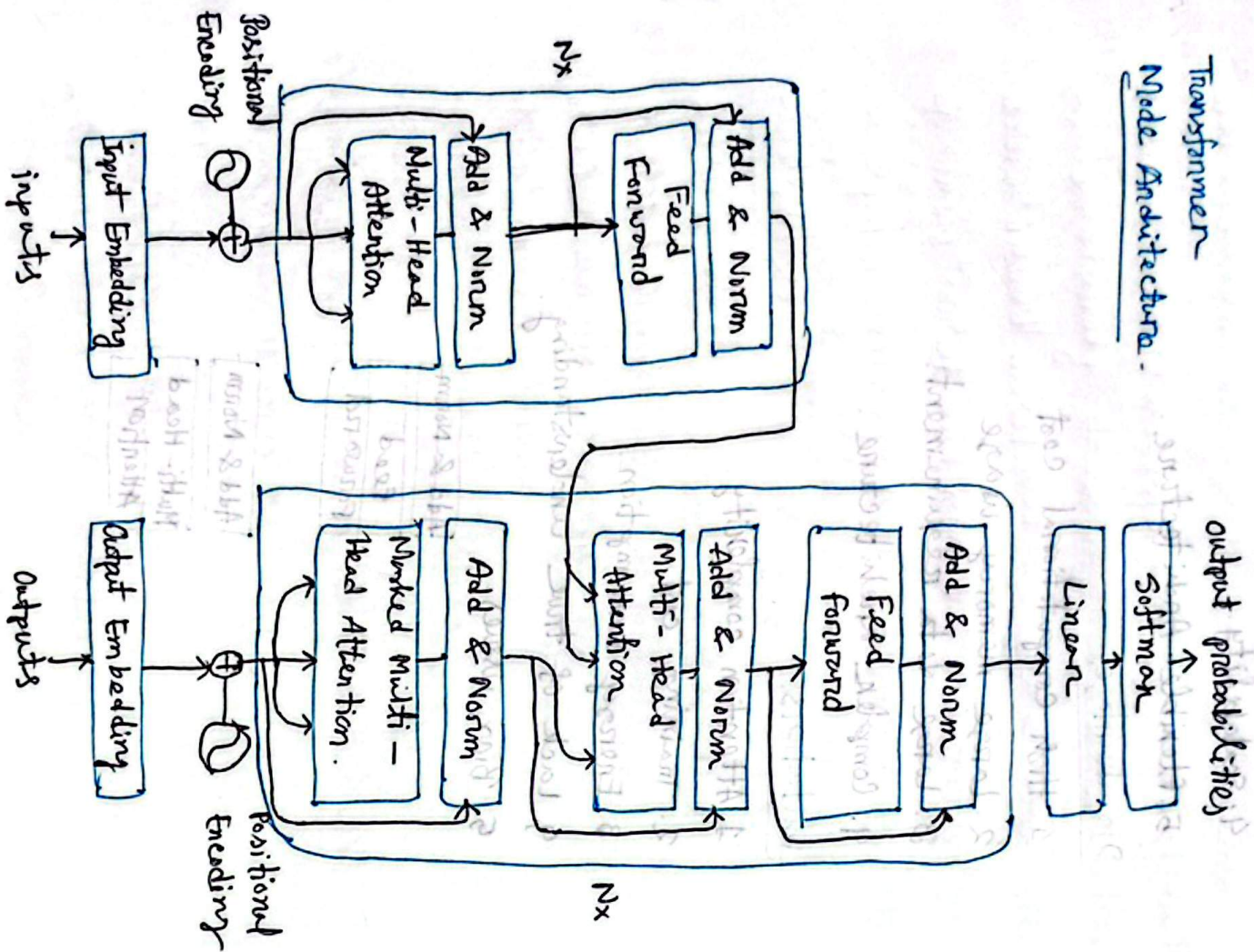
1. High computational cost.
2. Large memory usage.
3. Large data requirement.
4. Complex architecture.

Limitations:

1. Attention complexity.
2. Training cost.
3. Energy consumption.
4. Lack of true understanding.
5. Bias issues.



Transformer Node Architecture.



BERT Model (Bidirectional Encoder Representations) from Transformers

BERT is a Transformer-based language model developed by Google in 2018. It reads text bidirectionally (both left-to-right and right-to-left) to understand the context of words more accurately.

Example!

"The bank is near the river."

BERT uses words before and after "bank" to understand that it refers to a riverbank, not a financial institution.

Application

- Sentiment analysis.
- Text classification.
- Question answering.
- Named Entity Recognition (NER).
- Chatbots and virtual assistants.
- Search engines (Google Search).
- Text summarization.

Advantages

1. Bidirectional understanding.
2. High accuracy.
3. Pre-trained model.
3. Strong context understanding.
5. Transfer learning.

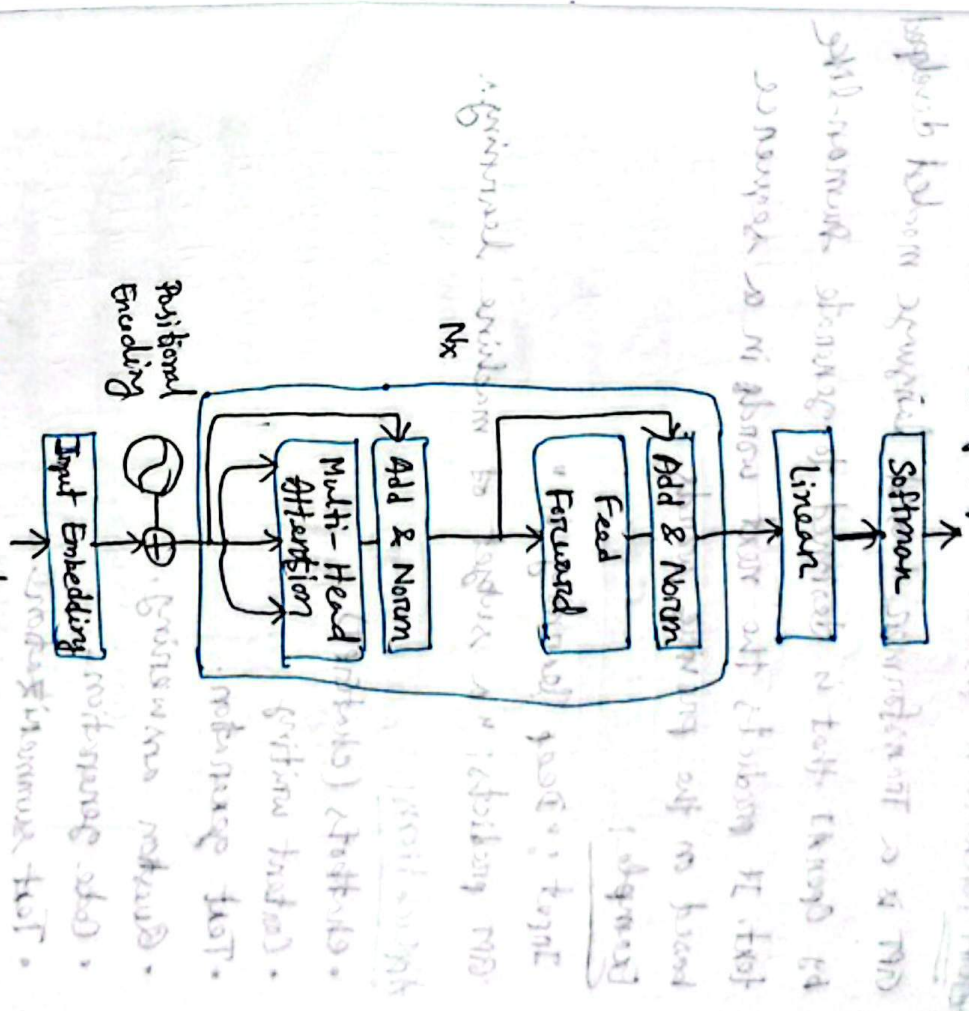
Disadvantages

1. Large Model size and slow down the TRN
2. Slow inference speed for high throughput
3. High computational cost with limited hardware
4. Complex architecture.

Limitations

1. Not designed for text generation.
2. Input length restriction.
3. Resource intensive.
4. Data Bias.
5. Limited Real-world reasoning.

output probabilities



BERT architecture inputs

- Input tokens
- Segment tokens
- Positional tokens

GPT (Generative Pre-trained Transformer)

GPT is a Transformer-based language model developed by OpenAI that is designed to generate human-like text. It predicts the next words in a sequence based on the previous words.

Example:

Input: "Deep learning is"

GPT predicts: "a subset of machine learning."

Applications:

- Chatbots (ChatGPT)
- Content writing
- Text generation
- Question answering
- Code generation
- Text summarization
- Language translation
- Virtual assistants

Advantages:

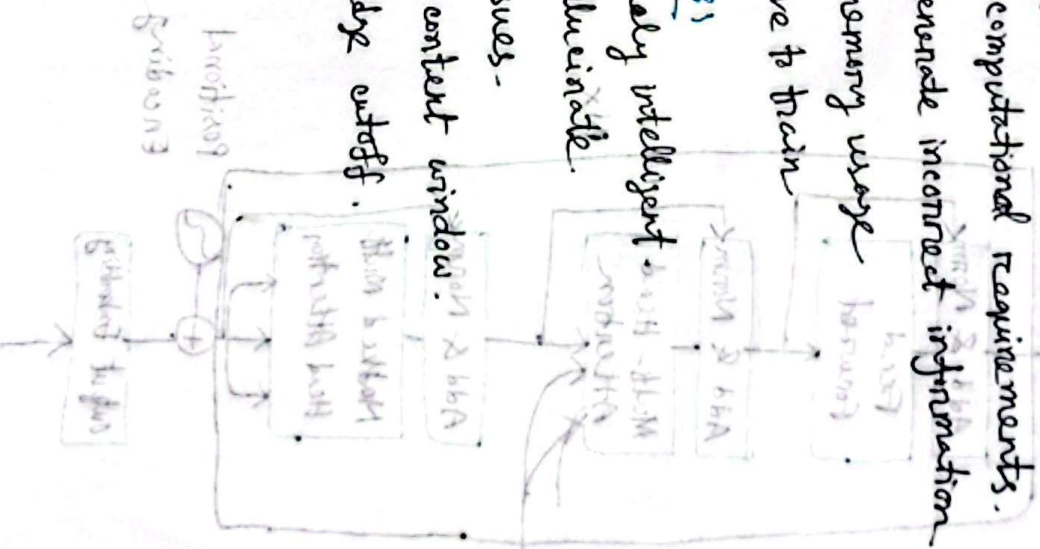
1. Excellent text generation.
2. Context awareness.

Disadvantages:

3. Versatile
4. Pre-trained knowledge
5. Supports few-shot learning.

Limitations:

1. Large computational requirements.
2. May generate incorrect information.
3. High memory usage.
4. Expensive to train.
5. Not truly intelligent.
6. Can hallucinate.
7. Bias issues.
8. Limited context window.
9. Knowledge cutoff.

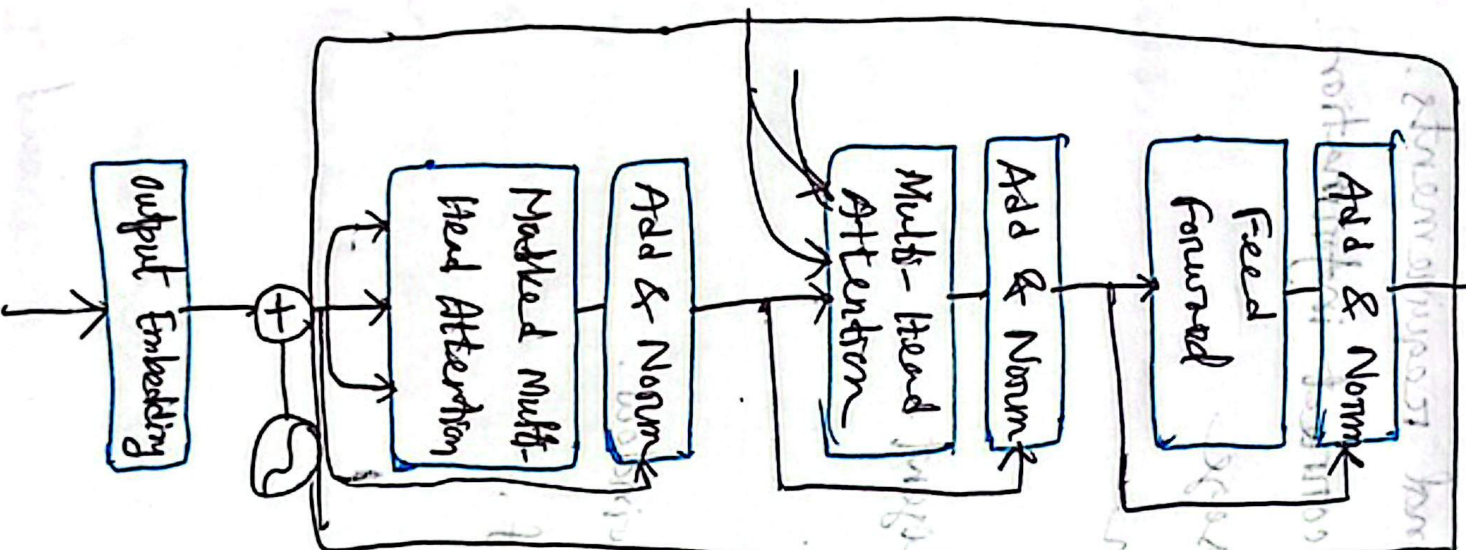


Output Probabilities

distraction

Softmax

Linear



Positional
Encoding

09.06.2020